

CUDA kernel fusion (CUDA 内核融合)：在 NVIDIA Hopper 架构上使用 CUTLASS (CUDA 线性代数模板库) 库实现 FlashAttention-2 (闪注意力-2) 的案例研究

我们提供了对 **FlashAttention-2 (闪注意力-2)** 前向传播过程的优化实现。FlashAttention-2 是一种流行的内存感知缩放点积注意力算法，我们的实现采用了针对 NVIDIA Hopper 架构的定制融合 CUDA 内核，并使用开源的 CUTLASS (CUDA 线性代数模板库) 库编写。在此过程中，我们讨论了将在线 softmax 与连续的 GEMM 内核融合时所面临的挑战和应用的技术，包括利用 Hopper 架构特有的张量内存加速器 (TMA) 和 Warpgroup 矩阵乘累加 (WGMMMA) 指令，定义和转换 CUTLASS 布局 (Layouts) 和张量 (Tensors)，重叠复制操作与 GEMM 操作，以及在平衡寄存器压力和共享内存使用的同时，为 Q、K、V 注意矩阵选择最佳的 tile 大小。在使用单个 H100 PCIe GPU 针对一些常见超参数设置进行的对比基准测试中，我们观察到相比于针对上一代 NVIDIA Ampere 架构优化的 FlashAttention-2 实现，我们的方法的 FLOPs/s (每秒浮点运算次数) 提高了 20%-50%。

1 引言

对于 GPU 上的并行编程，程序员可以使用的最强大且复杂的技术之一就是 **kernel fusion (内核融合)**，内核融合简单地指将多个独立的内核合并为一个新的单个内核。内核融合的主要优势在于减少对 **全局内存 (global memory)** 的读写次数，而全局内存是 GPU 内存层次结构中最大且速度最慢的级别。由于 GPU 的计算能力增长速度超过了内存带宽的增长速度，当代应用通常是 **内存受限 (memory-bound)** 而非 **计算受限 (compute-bound)**。因此，在密集型工作负载中，内核融合对打破所谓“**内存墙 (memory wall)**”的意义随着时间的推移不断增加。

当今最具挑战性的此类工作负载之一是 **大型语言模型 (LLMs)** 的训练和推理。现代 LLMs 通常是 Transformer 深度学习模型，拥有极大量的可学习参数；例如，GPT-3 约有 1750 亿个参数 [2]。在 Transformer 架构中，**注意力机制 (attention)** 是其中的核心 [3]。注意力计算包括两次矩阵乘法 and 一次逐行 **softmax** (row-wise softmax) 运算，这将在第2节中回顾。鉴于注意力在 Transformer 模型中的核心地位，它似乎是内核融合的一个天然候选操作。令人惊讶的是，据我们所知，直到2022年 Dao 等人才首次提出了将注意力作为融合内核的尝试，即 FlashAttention 算法 (FlashAttention algorithm) [4]，他们将其描述为一种注意力的“内存感知”版本。随后，Dao 在文献 [5] 中改进了该算法，并使用 NVIDIA 开源的 CUTLASS (CUDA 线性代数模板库) 库从头重写了该算法以实现高性能线性代数 [9, 10]。从此之后，FlashAttention 得到了广泛采用，并被视为当前的最先进技术 [7, 8]。

在本文及随附的代码中，我们以文献 [5] 中的 FlashAttention-2 作为内核融合的范例，研究将其实现为 CUDA 内核所涉及的工程挑战。与文献 [5] 一样，我们会大量利用 CUTLASS 库中的工具，该库通过布局 (Layouts) 和张量 (Tensors) 等抽象的系统使用极大地简化了 CUDA 内核的开发。由于本研究主要具有教学意义，我们将只限于注意力计算的前向传播 (forward pass) 部分。我们的目标有两个：

1. 实现一个专门针对 **Hopper (SM90) 架构** 的版本，利用 Hopper 特有的功能，例如用于数据复制的张量内存加速器 (TMA) 和用于 GEMM 的 Warpgroup 矩阵乘累加 (WGMMMA) 指令。相比之下，文献 [5] 中的实现针对的是 **Ampere (SM80) 架构**。

2. 记录并解释一些可能普遍适用于内核融合问题的挑战和技术，例如 CUTLASS 布局 (Layouts) 及其变换的重要性。我们的目的是通过强调文献【4,5】中所忽略的一些实现级别细节来补充那两篇论文的讨论。

附带的代码可以在 <https://github.com/ColfaxResearch/cutlass-kernels/tree/master/src/fmha> 获取。我们强烈建议读者在阅读本文的同时，运行并研究该代码。

2 标准注意力和 Flash（内存感知）注意力

本节快速回顾 Transformer 模型中的注意力机制以及 FlashAttention-2 算法。Transformer 模型的输入是一批形状为 $(L = \text{batch_size}, N = \text{seq_len})$ 的标记序列。嵌入层将此输入转换成形状为 $(L, N, D = \text{embedding_dim})$ 的张量 M 。然后，我们通过将 M 与三个可训练的方阵权重矩阵分别相乘（作为批矩阵乘法）来得到三个张量 Q 、 K 和 V 。接着，将 Q 、 K 、 V 在 D 维度上划分为 h 个“头”（head），每个头的维度为 $d = D/h$ ，因此得到的 Q, K, V 三个张量的形状为 $(L = \text{batch_size}, N = \text{seq_len}, h = \text{\#heads}, d = \text{headdim})$ 。每个头是独立训练和推理的。 h 和 d 的选择取决于模型，但通常情况下 $N \gg d$ 。例如，distilbert-base-uncased 模型【20】默认使用 $d = 64$ 、 $h = 12$ 和 $N = 512$ 。

我们滥用符号记法，让 Q 、 K 和 V 同样表示与某一给定头相关的 $N \times d$ 矩阵。那么注意力输出由下列公式给出：

$$O = \text{softmax}((1/\sqrt{d}) Q K^T) V = P V$$

其中 $S = (1/\sqrt{d}) Q K^T$ 且 $P = \text{softmax}(S)$ 是中间表达的常用变量名。在实际运算中，我们在进行 softmax 之前用 S 替换成 $S - \text{rowmax}(S)$ 来避免指数函数溢出；这并不改变 softmax 的输出结果。对所有头和批次进行拼接会生成送入模型后续层的输出张量。注意， O 的计算在不同的头和批次之间是独立的，因此可以并行执行。**GEMM（通用矩阵乘法）** 在行和列两个方向上也具有天然的并行特性【1】。考虑到这些，我们可以在 GPU 上给出以下朴素（或标准）的注意力实现算法：

算法 1. 标准注意力

1. 从高带宽内存 (HBM，高带宽存储器) 按块加载 Q 和 K 。
2. 计算 $S = (1/\sqrt{d}) Q K^T$ （GEMM-I）。
3. 将 S 写入 HBM。
4. 从 HBM 读取 S 。
5. 计算 $S = S - \text{rowmax}(S)$ 。
6. 计算 $P = \text{softmax}(S)$ 。
7. 将 P 写入 HBM。
8. 从 HBM 按块加载 P 和 V 。
9. 计算 $O = P V$ （GEMM-II）。
10. 将 O 写入 HBM。

将矩阵 S 和 P 材料化到 HBM（即全局内存）会对整体运行时长和内存需求产生负面影响。事实上，矩阵 S 的大小随着序列长度 N 的增加而成平方增长，而 N 值通常很大（例如，对于最先进的 LLM 模型，其 N 可能达到 4K 或甚至 32K）。因此，我们希望将注意力操作的各个步骤融合到一个单一的 CUDA 内核中，从而绕过对全局内存 (gmem) 的中间写操作。将 GEMM 与逐元素操作融合非常直接。相比之下，将 softmax 与 GEMM 融合就

不容易了，因为 softmax 预先需要沿 S 的行计算全局最大值和求和，而融合操作应在一个线程块 (即 CTA) 级别进行。

文献【5】中的融合多头注意力 (FMHA) 算法借鉴了在线 softmax (online-softmax) 算法【19】的思路，重构了注意力计算来克服这些困难并成功完成了融合。具体来说，FMHA 算法对矩阵 Q 和 K 进行分块 (tiling)，并对 GEMM-I 的输出执行“部分”或“局部”的 softmax，将结果存储在本地内存 (smem 或 rmem) 中。在使用 V 的分块进行 GEMM-II 时，从本地内存读取部分结果，使用缺失的缩放因子对它们重新缩放，然后将其累加到输出结果中。

FMHA 算法在算法 2 和图1中进行了回顾。我们已为 Q 和 K, V 选定块大小，即令 Q 的块大小 $b_M = QBLK$ ， K 和 V 的块大小 $b_N = KBLK$ ，使得 Q, K, V 沿行维度 (即 M 或 N 维度) 被分割为多个块，同时保持 K 维度不分块 ($b_K = d$)。我们还选择展示一种变体，其中用于第二次 GEMM 的第一个操作数存储在寄存器内存 (rmem) 中，而不是共享内存 (smem) 中。

算法 2. FlashAttention-2 (FMHA)

1. 对于 Q 的每个分块 (tiles of Q)，执行：
2. 从 HBM 加载一个 $b_M \times d$ 的 Q_i 分块到共享内存 (SMEM)。
3. 初始化大小为 $b_M \times d$ 的累加器 $O_i = 0$ 。
4. 将 $b_M \times 2$ 的行最大值向量 m_i 初始化为 $(-\infty)$ ，将 $b_M \times 1$ 的行累加和向量 Σ_i 初始化为 0。
5. 对于 K 的每个分块 (tiles of K)，执行：
6. 从 HBM 加载一个 $b_N \times d$ 的 K_j 分块到 SMEM。
7. 计算 $S_{ij} = (1/\sqrt{d})(Q_i K_j^T)$ (SS-GEMM-I)。
8. 更新行最大值 $m_i = (m_i^{\text{new}}, m_i^{\text{old}})$ ，跟踪步骤 j 和 $j-1$ 时的行最大值。
9. 计算 $P_{ij} = \exp(S_{ij} - m_i^{\text{new}})$ 。
10. 更新 $\Sigma_i = \exp(m_i^{\text{old}} - m_i^{\text{new}}) \Sigma_i + \text{rowsum}(P_{ij})$ 。
11. 从 HBM 加载一个 $b_N \times d$ 的 V_j 分块到 SMEM。
12. 计算 $O_i = \exp(m_i^{\text{old}} - m_i^{\text{new}}) O_i + P_{ij} V_j$ (RS-GEMM-II)。
13. 结束上述 K 分块循环。
14. 计算 $O_i = (1/\Sigma_i) O_i$ 。
15. 将 O_i 写入 HBM。
16. 结束上述 Q 分块循环。

累加器 O_i ， S_{ij} ， m_i 和 Σ_i 隐式地存储在寄存器内存 (rmem) 中。注意，对于外层循环而言，该算法在不同的线程块之间并行执行；而在外层循环内，算法在单个线程块内执行。因此，在代码实现中，内层循环会作为主要计算循环。

3 CUTLASS/CuTe 基础知识、TMA 和 WGMMA

NVIDIA 的开源库 CUTLASS 及其后端核心库 CuTe 使得高效编写针对 Hopper (SM90) 架构的融合 CUDA 内核成为可能。在本节中，我们将介绍在实现算法 2 为 CUDA 内核时需要使用的 CUTLASS/CuTe 中的抽象和方法，包括使用 Hopper 特有的 TMA (张量内存加速器) 和 WGMMA (Warpgroup 矩阵乘累加) 指令的异步拷贝和 GEMM 操作。

严格来说，FlashAttention-2 是 FMHA 算法的一个例子，但在本文中我们为了简便也将其称为 FMHA。这里的维度符号 K (在 GEMM 中作为内部维度) 与我们所用的矩阵 K 名称不要混淆。

图1. FMHA 算法的各计算阶段。smem 和 rmem 用于各阶段之间的存储。输出在最后阶段写回 gmem。图中未显示跨迭代对 S 和 O 应用的缩放。

3.1 布局 (Layouts) 和张量 (Tensors)

CuTe 的核心抽象是**布局 (Layout)**【11】。数学上，一个布局 $L = n:d$ 是由两个具有相同长度的整数元组构成的对象，其中 $\text{shape } n = (n_1, \dots, n_s)$ 和 $\text{stride } d = (d_1, \dots, d_s)$ 确定了一个多线性函数 $g: [0, n_1) \times \dots \times [0, n_s) \rightarrow \mathbb{N}, \quad (a_1, \dots, a_s) \mapsto \sum_{i=1}^s a_i d_i$ 等价地，这可以定义为单一变量函数 $f = g \circ \iota: [0, \prod_i n_i) \rightarrow \mathbb{N}$ ，其中 ι 表示按**列主序**遍历的同构映射。例如，二维“列主序”布局 $L=(4,4):(1,4)$ 对应的函数为 $f:[0,16) \rightarrow \mathbb{N}, \quad i \mapsto i$ ；而二维“行主序”布局 $L'=(4,4):(4,1)$ 对应的函数则将区间 $[0,16)$ 依次映射到集合 $\{0,4,8,12,1,5,9,13,2,6,10,14,3,7,11,15\}$ 上。布局用于指定从逻辑坐标（或坐标元组）到物理坐标的映射，例如到 HBM 或共享内存中的地址，或者（对于 MMA 指令）从一个 $(\text{线程}, \text{值})$ 坐标到一个矩阵坐标的映射。例如，对于 WGMMMA 指令，其 64×64 累加器 C 的布局在 `cute/atom/mma_traits_sm90_gmma.hpp` 中定义为：

```
using CLayout_64x64 = Layout<Shape<Shape<_4,_8,_4>,Shape<_2,_2,_8>>,
                             Stride<Stride<_128,_1,_16>,Stride<_64,_8,_512>>>;
```

该布局描述了 $(T,V) \mapsto (M,N)$ 的映射，对应于图2【18，第9.7.14节图118】中的示例。图2所示为 WGMMMA 累加器的结构示意图。CuTe 中的**张量 (Tensor)** 则由布局和指向内存的指针构造，或者可以通过对其他张量进行切分等方式得到（即 C++20 意义上的视图）。张量可以是“拥有的”（例如存储在寄存器中）或“非拥有的”（例如全局或共享内存中的视图）【12】。

尽管这些抽象乍看可能很复杂，但理解布局和张量的基本原理对开发融合内核至关重要。例如，我们将在后续看到，需要将 GEMM-I 的累加器布局重塑为 GEMM-II 操作数布局的情形。

3.2 TMA 复制

Hopper 引入了专用的**张量内存加速器 (TMA)** 单元，用于从全局内存到共享内存的异步复制。CUTLASS 通过 `make_tma_copy` 接口来支持 TMA，这需要给定完整的全局内存 Tensor 和目标共享内存布局。例如，下面的代码展示了为张量 Q 构造 TMA 复制对象的方法（见列表2）：

```
auto tileShapeQ = make_shape(bM{}, bK{});
auto smemLayoutQ = tile_to_shape(GMMA::Layout_K_SW128_Atom<MmaA>{}, tileShapeQ);
Layout gmemLayoutQ = make_layout(make_shape(M, K, H, B), make_stride(K * H, 1, K, H * M * K));
Tensor gQ = make_tensor(ptrQ, gmemLayoutQ);
auto tmaQ = make_tma_copy(SM90_TMA_LOAD{}, gQ, smemLayoutQ, tileShapeQ,
                          Int<1>{});
```

列表2：为 Q 构造 TMA 复制对象的代码示例。

为了获得最佳性能，我们为 `smemLayoutQ` 选择了以 `$K$` 为主序（即行主序）的 128 字节交织格式；由于 CuTe 已经预置了原子 (atom)，我们只需调用 `tile_to_shape` 以适应给定的块大小。共享内存中的交织是一种常见的 CUDA 优化技术，用于减轻银行冲突【22】。在设备代码中，我们可以使用 `tmaQ` 对象来进行实际的复制操作（列表3）：

```
cfk::copy(tQgQ(_, 0), tQsQ(_, 0), tmaLoadQ, tma_load_mbar[0]);
```

列表3：执行 TMA 复制的代码示例。这里 `cfk::copy` 是我们自定义的方法，将 `cute::copy` 包装在同步/屏障逻辑中。其他从全局内存到共享内存的复制操作也采用类似方式处理。

3.3 分块 MMA 和 GEMM

为了在 Hopper 上获得最佳性能，我们希望使用异步的 WGMMMA 指令来执行矩阵乘法。为此，CuTe 提供了 **MMA 原子 (MMA atom)**【13】及其包装类 **TiledMMA**。下面展示了为 GEMM-I 和 GEMM-II 选择相应 MMA 原子的示例代码（列表4）：

```
// 使用支持 SS 模式的 GMMMA 原子用于 GEMM-I
using TiledMma0 = decltype(cute::make_tiled_mma(
    cute::GMMMA::ss_op_selector<MmaA, MmaB, MmaC, Shape<bM, bN, bK>>(),
    MmaTileShape{}));
// 使用支持 RS 模式的 GMMMA 原子用于 GEMM-II（默认）
using TiledMma1 = decltype(cute::make_tiled_mma(
    cute::GMMMA::rs_op_selector<MmaA, MmaB, MmaC, Shape<bM, bK, bN>,
    GMMMA::Major::K, GMMMA::Major::MN>(),
    MmaTileShape{}));
```

列表4：用于 GEMM-I 和 GEMM-II 的分块 MMA (TiledMMA) 定义示例。

其中，`ss_op_selector` 和 `rs_op_selector` 是 CuTe 的辅助函数【13】，用于根据目标块大小、精度格式（操作数类型 `MmaA` 和 `MmaB` 为 FP16，累加器类型 `MmaC` 为 FP32），以及是否将第一个操作数存储在共享内存 (smem)（如 GEMM-I）或寄存器内存 (rmem)（如 GEMM-II）来选择合适的 SM90 MMA 原子。然后使用这些 TiledMMA 对象来构造在 GEMM 调用中使用的张量。例如，对于 GEMM-I，我们使用如下代码来准备参数（列表5）：

```
TiledMma0 tiledMma0;
auto threadMma0 = tiledMma0.get_thread_slice(threadIdx.x);
Tensor tSrQ = threadMma0.partition_fragment_A(sQ);
Tensor tSrK = threadMma0.partition_fragment_B(sK);
Tensor tSrS = partition_fragment_C(tiledMma0, tileShapeS);
// ...
cfk::gemm_bar_wait(tiledMma0, tSrQ, tSrK, tSrS, tma_load_mbar[0]);
```

列表5：GEMM-I 的代码示例。这里 `cfk::gemm_bar_wait` 包装了 `cute::gemm` 调用，并包含一些同步/屏障逻辑。

对于 GEMM-II，则对应如下代码（列表6）：

```
cfk::gemm_bar_wait(tiledMma1,
    convert_type<PrecType, AccumType>(tOrP),
    tOrV,
    tOrO,
    tma_load_mbar[1]);
```

列表6：GEMM-II 的 `gemm` 调用示例，执行在主循环中。

4 布局变换

正确地为 GEMM-II 的操作数张量 `tOrP` 和 `tOrV` 定义布局需要进行一些特殊的变换，本节对此进行讨论。

4.1 转置布局

在 CuTe 中，默认的 GEMM 调用采用 BLAS NT 格式，即 `cute::gemm` 将 $M \times K$ 矩阵 A 和 $N \times K$ 矩阵 B 作为输入，并计算 $C = A B^T$ 。回顾一下，GEMM-I 计算 $Q K^T$ ，而 GEMM-II 计算 $P V$ 。因此，我们需要将 V 的布局（在共享内存中）变换为适合 GEMM-II 第二操作数的布局。假设 V 在共享内存中的形状为 (bN, bK) ，我们可以通过前置构造得到转置布局：

```
auto tileShapeV = make_shape(bN{}, bK{});
auto smemLayoutV = tile_to_shape(GMMA::Layout_K_SW128_Atom<MmaB>{}, tileShapeV);
// 为 V^T 创建布局，用于 GEMM-II
auto tileShapeVt = make_shape(bK{}, bN{});
auto smemLayoutVt = composition(smemLayoutV, make_layout(tileShapeVt,
    GenRowMajor{}));
```

列表7：为 V 的转置创建布局的示例代码。

在数学上，给定两个布局 L 和 L' 对应的函数【14】 $f, f': \mathbb{N} \rightarrow \mathbb{N}$ ，我们可以形成复合函数 $f \circ f'$ ，并判断是否存在一个布局 L'' 使得它的关联函数 $f'' = f \circ f'$ ；如果存在，则称 $L \circ L' = L''$ 。CuTe 的 `composition` 函数可以自动为程序员求解出 L'' 【15】。在当前情况下，我们通过将 `smemLayoutV` 与布局 $(b_K, b_N):(b_N, 1)$ 前置组合来得到转置布局【16】。注意原始的 `smemLayoutV` 布局已经后置了一个交织函数，这个前置操作不会影响该交织。

由于从全局内存到共享内存的拷贝是基于原布局 `smemLayoutV` 完成的，我们可以据此构造一个在共享内存中访问数据的转置 Tensor `sVt`，并正确地定义 `tOrV`：

```
Tensor sVt = make_tensor(make_smem_ptr(shared_storage.smem_v.data()),
    smemLayoutVt);
// ...
Tensor tOrV = threadMma1.partition_fragment_B(sVt);
```

列表8：构造 `sVt` 和定义 `tOrV` 的示例代码。

4.2 将累加器布局变换为操作数布局

对于 GEMM-II 中的张量 `tOrP` 的布局定义，我们使用了一个自定义的布局变换方法 `ReshapeTStoTP`。该方法接受累加器 Tensor `tSrS` 和由 GEMM-II 的 TiledMMA 对象给出的 Tensor `tOrS`，并返回一个适合定义 `tOrP` 的新布局。例如，我们有：

```
Tensor tOrS = threadMma1.partition_fragment_A(sS);
auto tOrPLayout = ReshapeTStoTP()(tSrS, tOrS);
auto tOrP = make_tensor(tSrS.data(), tOrPLayout);
```

列表9：通过 `ReshapeTStoTP` 将累加器布局转换为操作数布局的示例代码。

这样的布局变换是必要的，因为我们需要按照第二次 GEMM 操作数的 A 布局来遍历第一次 GEMM 的累加器 `sC`（这些值存储在寄存器中）。由于我们明确选择了使得 GEMM-II 操作数 `sA` 存储在寄存器的 MMA 原子，相关的 A 布局示例如下（取 16 位值类型）：

```
using ALayout_64x16 = CLayout_64x16;
using CLayout_64x16 = Layout<Shape<Shape<_4,_8,_4>,Shape<_2,_2,_2>>,
    Stride<Stride<_128,_1,_16>,Stride<_64,_8,_512>>>;
```

以上布局与图2中 `sC` 的布局相同，只不过操作数尺寸固定为 64×16 ，意味着每个 128 线程 warpgroup 每线程有 8 个值。为了说明这一点，考虑块大小 $(bM, bN, bK) = (128, 128, 128)$ 的情形。此时 GEMM-I 的累加器布局为 `CLayout_64x128`。在控制台打印会显示：

```
tSrS: ptr[32b] ... (布局与 CLayout_64x128 对应)
tOrS: ptr[16b] ... (布局与 ALayout_64x16 对应)
tOrPLayout: ...
```

这些布局描述了一个从逻辑坐标到寄存器地址的映射，其中寄存器内存被视为物理地址。可以看到，对于 `tSrS` 和 `tOrS`，它们的关联函数（作为一维映射）实际上是恒等映射【17】；我们对给定线程的值按连续顺序存储在寄存器内，并按列主序分块。 64×16 与 64×128 区别在于行维度不同。因此，在定义 `tOrP` 时，我们需要以不同于 `tOrS` 布局所定义的顺序来遍历 `tSrS.data()`，这导致 `tOrPLayout` 中的步幅与 `tOrS` 不同。相反，如果 $bM=64$ ，则无需进行这种重塑：此时 `tSrS` 和 `tOrS` 的数据布局天然一致。

5 在线 Softmax 和 Shuffle 规约

算法 2 内循环中 GEMM-I 和 GEMM-II 之间的 **在线 softmax** 部分涉及对矩阵 S 的逐行最大值和累加和进行计算，并就地修改 S 矩阵。此外，在循环的第一次迭代之后，我们需要连续地重新缩放矩阵 O 。在代码中，我们如下实现在线 softmax（列表9）：

```
if (blockIdxY == 0) {
    // Compute Online Softmax and NO Output Rescaling.
    onlineSoftmaxAndRescale<true, AccumType>(rowMax, rowSum, tSrS, tOrO, scale);
} else {
    // Compute Online Softmax and Output Rescaling.
    onlineSoftmaxAndRescale<false, AccumType>(rowMax, rowSum, tSrS, tOrO,
scale);
}
```

列表9：主循环中的在线 softmax 计算代码。 `blockIdxY == 0` 时不重新缩放输出，其他情况执行重新缩放。

GEMM-I 之后，矩阵 S 的元素如图2所示驻留在每个线程的寄存器中。具体地，每个线程对应两行值，并以复制的“Z”形模式进行遍历；例如，见列表7-8。在代码中，这意味着我们需要维护两个行最大值，当遍历寄存器内存 `tSrS.data()` 时同时更新它们（列表10）：

```
for (int k = 0; k < NT * size<2>(VT); ++k) {
    data[n] = FragValType(AccumType(data[n]) * scaleFactor);
    max0 = cutlass::fast_max(max0, AccumType(data[n])); n++;
    data[n] = FragValType(AccumType(data[n]) * scaleFactor);
    max0 = cutlass::fast_max(max0, AccumType(data[n])); n++;
    data[n] = FragValType(AccumType(data[n]) * scaleFactor);
    max1 = cutlass::fast_max(max1, AccumType(data[n])); n++;
    data[n] = FragValType(AccumType(data[n]) * scaleFactor);
    max1 = cutlass::fast_max(max1, AccumType(data[n])); n++;
}
```

列表10：对 `tSrS.data()`（寄存器内存中的数据）进行缩放并计算线程级行最大值的代码示例。

从图2我们还可以看到，一行的元素由4个线程（一个线程组）共同处理。为了将这些线程级的行最大值组合成全局的行最大值，我们可以使用原子最大运算。然而，NVIDIA 提供了从 Kepler 架构开始用于线程间数据交换的 **shuffle** 指令【23】。我们可以利用这些指令来避免原子操作，从而避免因原子操作带来的内存访问延迟（列表11）：

```
auto max_quad_0 = ShflReduce<4>::run(max0, maxOp);
auto max_quad_1 = ShflReduce<4>::run(max1, maxOp);
```

```
mi(rowId) = max_quad_0;
mi(rowId + 1) = max_quad_1;
```

列表11：利用 *shuffle* 指令对两行的最大值进行两次规约操作，从而计算出全局行最大值。

`ShflReduce` 方法是一个典型的实现（列表12），此处附上以便完整性：

```
template <typename T> struct MaxOp {
    __device__ inline T operator()(T const &x, T const &y) {
        return x > y ? x : y;
    }
};

template <int THREADS> struct ShflReduce {
    static_assert(THREADS == 32 || THREADS == 16 || THREADS == 8 || THREADS ==
4);
    template <typename T, typename Operator>
    static __device__ inline T run(T x, Operator &op) {
        constexpr int OFFSET = THREADS / 2;
        x = op(x, __shfl_xor_sync(uint32_t(-1), x, OFFSET));
        return ShflReduce<OFFSET>::run(x, op);
    }
};

template <> struct ShflReduce<2> {
    template <typename T, typename Operator>
    static __device__ inline T run(T x, Operator &op) {
        x = op(x, __shfl_xor_sync(uint32_t(-1), x, 1));
        return x;
    }
};
```

列表12：利用 *shuffle* 指令进行全局行最大值计算的示例代码。`ShflReduce<4>::run` 对一个四线程组计算全局最大值。

行累加和的计算过程与行最大值类似，也使用了分步累加后再通过 *shuffle* 来汇总。

6 复制与 GEMM 的重叠

Hopper GPU 编程的一个重要特性是能够将异步的 TMA 复制与异步的 WGMMMA 指令重叠执行，以隐藏内存延迟并最大化 GPU 吞吐量。为此，CUTLASS 建议使用带有多个缓冲区的软件流水线来对每个阶段进行流水线并行

【14】。从软件工程的角度来看，实现这样的方案代价很高，因为它需要对代码结构进行大规模修改。此外，这还会显著增加对共享内存的需求。我们可以利用 FMHA 算法的结构，其中内循环中包含两个 GEMM。我们不为单个 GEMM 构建独立流水线，而是将 GEMM-II 的加载操作与 GEMM-I 同时发起，反之亦然。产生的代码示例如下（列表13）：

```

// 预先将 K 的第一个分块从全局内存复制到共享内存。
cfk::copy_nobar(tKgK(_, 0), tKsK(_, 0), tmaLoadK, tma_load_mbar[0]);

for (uint64_t blockIdxY = 0; blockIdxY < nTilesOfK; ++blockIdxY) {
    // ...
    // 将当前的 V 分块从全局内存复制到共享内存。
    cfk::copy_nobar(tVgV(_, 0), tVsV(_, 0), tmaLoadV, tma_load_mbar[1]);
    clear(tSrS);

    // 发起 GEMM-I。
    cfk::gemm_bar_wait(tiledMma0, tSrQ, tSrK, tSrS, tma_load_mbar[0]);
    // ...

    // 将下一个 K 分块从全局内存复制到共享内存。
    if (blockIdxY != (nTilesOfK - 1)) {
        // ...
        cfk::copy_nobar(tKgK(_, 0), tKsK(_, 0), tmaLoadK, tma_load_mbar[0]);
    }
    // ...

    // 发起 GEMM-II，操作数 A 来自寄存器内存。
    cfk::gemm_bar_wait(tiledMma1,
        convert_type<PrecType, AccumType>(tOrP),
        tOrV,
        tOrO,
        tma_load_mbar[1]);
}

```

列表13：使用 TMA+WGMMA 实现复制与 GEMM 重叠的代码片段。其中 `cfk::copy_nobar` 表示在不使用额外屏障的情况下执行异步复制。该循环同时交错执行 GEMM-I 和 GEMM-II。

7 实验结果

我们将自己实现的 FMHA 前向传播与另外两个版本进行了对比基准测试：一个是随 CUTLASS 3.3 提供的示例 FMHA 内核【10】，另一个是 Dao AI Lab 的 FLASH-2 实现【6】。这两种版本都使用 SM80 ISA 来执行复制和 GEMM，而我们的实现使用 SM90 ISA（支持 TMA 和 WGMMA）。我们所有的实验都在一块带 PCIe 的 H100 GPU 上进行，并在图3中总结了结果。为正确解读这些结果，读者应注意以下几点：

- 我们的 COLFAX 内核使用 CUTLASS 3.3 和 CUDA 12.2 编译。
- CUTLASS FMHA 内核作为示例41出现在 CUTLASS 库中，并源自 xFormers【17】。我们使用了 FLASH-2 2.3.2 发布版中的实现。
- 在我们的测试中，我们针对不同的头维度，对 QBLK 和 KBLK 在 $(64,128) \times (64,128)$ 范围内进行了测试，并选择了表现最佳的组合（见表1）。
- 输入矩阵元素为符合均值0方差1的高斯分布随机值（类似层归一化的输出）。

- 与文献【5，第4.1节】一致，我们计算浮点运算量时仅考虑两个矩阵乘法的主要贡献，忽略 softmax 等低阶开销。
- 所有三个 CUDA 内核均使用了 NVCC 的 `--use_fast_math` 编译选项。
- 操作数类型为 FP16，累加器类型为 FP32。
- 我们对不同的内核进行了大量迭代（迭代次数=1000）的测试。重复基准时，我们观察到性能可能波动约 1 TFLOPs/s。

头维度 (head_dim)	QBLK×KBLK = 64×64	64×128	128×64	128×128
64	230.1	259.5	247.9	251.4
128	292.6	289.3	295.7	208.7
256	308.1	276.1	39.3	36.7

表1. 不同 QBLK×KBLK 大小下的性能 (单位: TFLOPs/s)。
数据为 H100 PCIe GPU、batch_size=4、head_dim 所示的情况。

从表中可以看出，我们的方法相较于 CUTLASS 有近 **2.5-3×** 的加速，但相比 FLASH-2 只有 **20%-50%** 的性能提升。根据之前对 GEMM 的实验经验【1】，我们原本预期 128×128 的块大小会有最好的性能表现。然而， 128×128 的配置由于寄存器压力问题而性能下降。在使用 NVCC 编译时，我们在 128×128 场景下观察到了寄存器溢出。此外，在 GEMM-II 中，由于操作数 A 和累加器 C 都需要存储在寄存器内存 (rmem) 中，而寄存器数量有限，无法同时保持它们，这导致了 GEMM-II 的串行化问题。

性能最高的版本每个 CTA 使用一个 warpgroup (128 线程)，这样会有一半的寄存器空间未被使用【21】。在研究过程中，我们尝试让每个 CTA 使用两个 warpgroup (256 线程) 来提高并行度。虽然这增加了每个 CTA 的寄存器需求，但实验结果却显示性能下降，因此我们没有在本文中报告该方案。我们正在进行使用两个 warpgroup 的更优实现的研究。

图3. 在 Hopper (H100 PCIe) GPU 上 FMHA (前向传播) 的性能对比。序列长度 $seq_len=key_len=4096$ ，head_dim=[64,128,256]，head 数=[32,16,8]，batch_size=4，使用 FP16 精度和 FP32 累加器。CUTLASS 和 FLASH-2 版本针对 GEMM/COPY 使用 SM80 ISA；本工作采用的版本使用 SM90 ISA。

8 未来工作

本工作最初是更大范围的 CUDA 优化技术研究的一部分，重点考察了迁移到 Hopper 架构后所提供的新功能。从 NVIDIA H100 数据表【15】中我们了解到，FP16 Tensor Core 的理论峰值为 756 TFLOPs/s【22】。因此，我们相信针对注意力问题的可行优化远未穷尽。我们计划在后续工作中研究以下优化方向：

- 在每个 CTA 中使用两个 warpgroup (256 线程) 并采用适当的 warp 专门化 (WS) 方案；
- 在复制-计算重叠策略中引入更多流水线阶段；
- 利用线程块集群和用于 K 和 V 矩阵复制的新型分布式共享内存。

考虑到 NVIDIA 已经在 LLM 生产库中实现了 SM90 版本的 FMHA（包括前向和反向过程），例如 TensorRT-LLM 中采用了 TMA 和 warp 专门化的 FMHA 内核【21】，以及 OpenAI 的 Triton 最新构建中也包含了 SM90 版本，上述

研究具有重要意义。未来，随着 GPU 架构的进一步演进，我们也将继续探索如何针对新硬件优化 FMHA 内核的设计。

参考文献

- [1] Developing CUDA Kernels for Accelerated Matrix Multiplication on NVIDIA Hopper Architecture using the CUTLASS Library. Colfax Research, 2023. <https://research.colfax-intl.com/nvidia-hopper-gemm-cutlass/>
- [2] Tom B. Brown et al. *Language Models are Few-Shot Learners*. May 28, 2020. <https://arxiv.org/abs/2005.14165>
- [3] Ashish Vaswani et al. *Attention Is All You Need*. August 2, 2023. <https://arxiv.org/abs/1706.03762>
- [4] Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. June 23, 2022. <https://arxiv.org/abs/2205.14135>
- [5] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. July 17, 2023. <https://arxiv.org/abs/2307.08691>
- [6] Dao-AILab. *FlashAttention — Fast and memory-efficient exact attention*. <https://github.com/Dao-AILab/flash-attention>
- [7] Dao-AILab. *FlashAttention adoption*. <https://github.com/Dao-AILab/flash-attention/blob/main/usage.md>
- [8] Ashraf Eassa and Sukru Burc Eryilmaz. *Setting New Records at Data Center Scale Using NVIDIA H100 GPUs and NVIDIA Quantum-2 InfiniBand*. November 8, 2023. <https://developer.nvidia.com/blog/setting-new-records-at-data-center-scale-using-nvidia-h100-gpus-and-quantum-2-infiniband/>
- [9] Andrew Kerr, Duane Merrill, Julien Demouth, John Tran. *CUTLASS: Fast Linear Algebra in CUDA C++*. December 5, 2017. <https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/>
- [10] NVIDIA CUTLASS. *CUDA Templates for Linear Algebra Subroutines*. <https://github.com/NVIDIA/cutlass>
- [11] NVIDIA CUTLASS. *CuTe Layouts*. https://github.com/NVIDIA/cutlass/blob/main/media/docs/cute/01_layout.md
- [12] NVIDIA CUTLASS. *CuTe Tensors*. https://github.com/NVIDIA/cutlass/blob/main/media/docs/cute/03_tensor.md
- [13] NVIDIA CUTLASS. *Support for Matrix Multiply-Accumulate instructions in CuTe*. https://github.com/NVIDIA/cutlass/blob/main/media/docs/cute/0t_mma_atom.md
- [14] NVIDIA CUTLASS. *Efficient GEMM in CUDA*. https://github.com/NVIDIA/cutlass/blob/main/media/docs/efficient_gemm.md
- [15] NVIDIA. *NVIDIA H100 Tensor Core GPU Datasheet*. <https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet>
- [16] NVIDIA. *Hopper Tuning Guide*. <https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html>
- [17] Benjamin Lefaudeux et al. *xFormers: A modular and hackable Transformer modelling library*. 2022. <https://github.com/facebookresearch/xformers>
- [18] NVIDIA. *Parallel Thread Execution (PTX) ISA Version 8.2*. <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>
- [19] Maxim Milakov and Natalia Gimelshein. *Online normalizer calculation for softmax*. July 28, 2018. <https://arxiv.org/abs/1805.02867>
- [20] Victor Sanh et al. *DistilBERT: a distilled version of BERT*. March 1, 2020. <https://arxiv.org/abs/1910.01108>
- [21] NVIDIA. *TensorRT-LLM*. <https://github.com/NVIDIA/TensorRT-LLM/tree/release/0.5.0>
- [22] Mark Harris. *Using Shared Memory in CUDA C/C++*. January 28, 2013. <https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/>
- [23] Justin Luitjens. *Faster Parallel Reductions on Kepler*. February 13, 2014. <https://developer.nvidia.com/blog/faster-parallel-reductions-kepler/>

[24] Dylan Patel. *How Nvidia's CUDA Monopoly In Machine Learning Is Breaking - OpenAI Triton And PyTorch 2.0*. January 16, 2023. <https://www.semianalysis.com/p/nvidiaopenaitritonpytorch>
